Agentic AI Architecture — Presenter Guide

Day 2 of *Arquitetura de Sistemas de IA*, 27 October 2026. English, topic-led, 40 minutes including discussion. No running example: each topic stands on its own paper. Real systems appear as evidence — **Pi**, **Pi Durable** and **Hermes**, plus the eleven harnesses of the source-code study.

Timing and ownership

Time	Presenter	Slides	Question
0–2 min	All	1–2	Opening and agenda
2–9 min	Ricardo	3–8	What is a harness?
9–15 min	Tomás	9–14	Which patterns describe an agent?
15–21 min	Simon	15–19	How do we evaluate its architecture?
21–29 min	Luca	20–26	What does context engineering look like?
29–37 min	Matheus	27–33	What changes when agents run always-on?
37–40 min	All	34	Discussion

About 70 seconds per slide. Each slide has a large figure, four key concepts and a one-sentence takeaway. The right-hand column is read by the audience; directions for the presenter are in this guide and in the speaker-notes PDF.

Presenting and editing

Arrow Right/Space reveals the next part of a diagram, then moves to the next slide; Arrow Left reverses. Page Down/Page Up change slides directly. G opens the contents; F toggles fullscreen. E toggles local text editing (Escape finishes; Ctrl/Cmd+S downloads an edited copy). Browser edits stay on that laptop. To change slide text for everyone, edit your file in `slides/day2/` and rebuild.

Boundaries of the evidence

- Paper numbers are quoted from the cited sections: eleven harnesses and 29 patterns (Barbaste et al.), 18 patterns from 57 studies (Liu et al.), eleven general scenarios and the Luna case (Lu et al.), 466 of 10,000 repositories and 155 AGENTS.md files (Mohsenimofidi et al.).

- Pi and Hermes details come from the harness study (July 2026 snapshot) and the projects' own documentation; Pi Durable from Earendil's announcement (1 October 2026). Pi Durable is experimental and its API may change.
- The session → durable → always-on spectrum and the scenario-to-harness table (slide 19) are **our synthesis**, labelled as such.
- The 69% token figure is one user's field report (Hermes issue #5563), not a benchmark.

1. Agentic AI Architecture

Section: Opening. **Presenter:** All presenters.

From a model in a pipeline to a model that acts.

Next: Five questions in forty minutes — All presenters.

2. Five questions in forty minutes

Section: Opening. **Presenter:** All presenters.

Each presenter answers one question, grounded in one paper.

Do: Read only the question column aloud; the times are for us.

- **Five topics**, each self-contained: one question, one main source.
- **Real systems as evidence:** Pi, Pi Durable, Hermes and the eleven harnesses of one source-code study.
- **Topic-led:** no single running example this time.
- **Discussion** reserved for the last three minutes.

Takeaway: Same question as Day 1, one layer up: what must surround the model?

Reading anchor: Group handoff · Day 2 outline.

Next: One layer up from Day 1 — Ricardo.

3. One layer up from Day 1

Section: Opening. **Presenter:** Ricardo.

Day 1 put a model inside a system. Day 2 puts it inside a loop that acts.

Builds: press Right 3 times to complete the figure.

Do: Point at the top lane first, then the bottom lane; finish on the return arrow.

- **Day 1:** a one-way path from records to a business decision.
- **Day 2:** the model chooses an action, not just a score.
- **The return arrow** is new: results flow back as context.
- **Same engineering question:** what must surround the model?

Takeaway: An agent is still a model inside an architecture — now one that loops.

Reading anchor: Day 1 lecture · Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §1.

Next: Agent = model + harness — Ricardo.

4. Agent = model + harness

Section: Topic 1 / Harness engineering. **Presenter:** Ricardo.

The harness is everything except the model. A source-code study of eleven harnesses found the same seven subsystems in all of them.

Builds: press Right 3 times to complete the figure.

Do: Start at the agent loop, then sweep across the six boxes; end on the size line at the bottom.

- **Agent loop** at the centre: model → tool → observation.
- **Six more subsystems:** LLM access, tools, safety, memory, orchestration, extensions.
- **Every harness takes a position** on all seven, even if it is “none”.
- **Interfaces** sit on top: terminal, editors, chat gateways.

Takeaway: The harness is the part you can engineer.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §2.3, Table 1.

Next: Eleven harnesses, from one tool to 109 — Ricardo.

5. Eleven harnesses, from one tool to 109

Section: Topic 1 / Harness engineering. **Presenter:** Ricardo.

The same seven subsystems, built with tool surfaces that differ by two orders of magnitude.

Builds: press Right 3 times to complete the figure.

Do: Reveal left to right: start at the single bash tool, end at Hermes and OpenClaw.

- **Minimal end:** Mini-SWE-Agent works through one bash tool; Pi exposes four by default.
- **Maximal end:** Hermes registers 69 tools and scopes them per platform; OpenClaw ships 109.
- **More code is not better results:** loop sophistication does not predict benchmark performance.
- **Most code goes elsewhere:** safety, user experience, extensibility and clients.

Takeaway: Tool count is a design philosophy, not a quality score.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §5 Table 4, §8.1 Fig. 3.

Next: The agent loop: decide, act, observe — Ricardo.

6. The agent loop: decide, act, observe

Section: Topic 1 / Harness engineering. **Presenter:** Ricardo.

One cycle repeats until the model answers in text instead of calling a tool.

Builds: press Right 3 times to complete the figure.

Do: Walk the ring clockwise from the model; point at the hub each time something is written back.

- **The context window accumulates state:** every pass writes into it.
- **Clockwise:** model → permission gate → tool → observation.
- **Steering input** waits for a safe boundary, never mid-tool-call.
- **Stop conditions differ:** no cap in Pi, a 90-step budget in Hermes.

Takeaway: A loop with a growing context — everything else manages that loop.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §6.

Next: Pi: a minimal core, everything else an extension — Ricardo.

7. Pi: a minimal core, everything else an extension

Section: Topic 1 / Harness engineering. **Presenter:** Ricardo.

Pi keeps the core small and moves safety, sub-agents and plan mode into extensions.

Builds: press Right 2 times to complete the figure.

Do: Read the four layers top to bottom, then point at the extension host.

- **Four layers:** interfaces → product → loop → model gateway.

- **Session tree:** JSONL entries you can fork and rewind.
- **Extension host:** sub-agents, plan mode and permissions hook into the event stream.
- **Trade-off:** a small core, but safety depends on what you install.

Takeaway: Minimal core plus extensions is one coherent harness architecture.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §10–12 · Zechner (2025); Pi docs, github.com/earendil-works/pi.

Next: What eleven harnesses share — and lack — Ricardo.

8. What eleven harnesses share — and lack

Section: Topic 1 / Harness engineering. **Presenter:** Ricardo.

Production harnesses converge on skills and context files, and all skip two popular technologies.

Builds: press Right 2 times to complete the figure.

Do: Pause on the two empty rows of dots. Then hand over to Tomás.

- **Shared:** skills, summarised history, Markdown context files, MCP.
- **Absent everywhere:** agentic frameworks and embedding search over code.
- **Instead:** hand-rolled async loops and deterministic search (grep, LSP).
- **Next:** a vocabulary for the agent inside the loop.

Takeaway: The field runs on simple loops and deterministic retrieval.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §13.2, Observation 8–9.

Next: A shared vocabulary for agent design — Tomás.

9. A shared vocabulary for agent design

Section: Topic 2 / Design patterns. **Presenter:** Tomás.

A systematic review of 57 studies distilled 18 architectural patterns for foundation-model agents.

Builds: press Right 3 times to complete the figure.

Do: Follow the user's goal from left to right before naming any pattern.

- **Inside the agent:** goal creation, planning, reflection.

- **Outside it:** the model, knowledge, a registry, tools and workers.
- **Each pattern** documents context, forces, solution, consequences, known uses.
- **Why it matters:** two teams can compare designs precisely.

Takeaway: Patterns let us name a design choice and its trade-off.

Reading anchor: Liu et al. (2025), Agent Design Pattern Catalogue, JSS 220, §2–4, Fig. 2.

Next: Eighteen patterns in eight families — Tomás.

10. Eighteen patterns in eight families

Section: Topic 2 / Design patterns. **Presenter:** Tomás.

Group the catalogue by the question each pattern answers.

Builds: press Right 2 times to complete the figure.

Do: Name the eight family titles only; point at the filled dots for the next three slides.

- **Eight families**, from goal creation to evaluation.
- **Most agents combine** several small patterns.
- **Deep dive on three:** planning, reflection, registry.
- **Coding harnesses skip RAG** and use grep and the language server instead.

Takeaway: Most agents combine many small patterns, not one big one.

Reading anchor: Liu et al. (2025), Agent Design Pattern Catalogue, JSS 220, Table 1 · Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §13.2.

Next: Planning: one path or many? — Tomás.

11. Planning: one path or many?

Section: Topic 2 / Design patterns. **Presenter:** Tomás.

A plan generator decomposes a goal; the choice is how many options it keeps open.

Builds: press Right 2 times to complete the figure.

Do: Top lane first, then the branching lane; ask which one a coding agent should use.

- **Single path:** one ordered list — cheap, simple, explainable.

- **Multi-path:** options per step, compared before choosing.
- **Trade-off:** multi-path improves certainty but costs more model calls.
- **A visible plan** lets people correct it early.

Takeaway: Most coding agents choose a single, visible plan.

Reading anchor: Liu et al. (2025), Agent Design Pattern Catalogue, JSS 220, §4.7–4.8.

Next: Reflection: who checks the work? — Tomás.

12. Reflection: who checks the work?

Section: Topic 2 / Design patterns. **Presenter:** Tomás.

Reflection patterns differ by who gives the feedback: the agent, another agent, or a person.

Builds: press Right 3 times to complete the figure.

Do: One loop at a time, from the inside out.

- **Self:** cheap and fast, limited by the agent's own blind spots.
- **Cross:** a second agent adds a perspective, and cost.
- **Human:** aligns with preferences, adds latency and contestability.
- **Real harnesses mix all three** at different moments.

Takeaway: Choose the reviewer by the cost of a wrong action.

Reading anchor: Liu et al. (2025), Agent Design Pattern Catalogue, JSS 220, §4.9–4.11 · Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §6.

Next: Registries and adapters: finding capabilities — Tomás.

13. Registries and adapters: finding capabilities

Section: Topic 2 / Design patterns. **Presenter:** Tomás.

A registry makes tools and agents discoverable; an adapter connects the loop to outside systems.

Builds: press Right 3 times to complete the figure.

Do: Follow the loop → registry → capability path, then the adapter path below.

- **Registry:** tool schemas, skills and agent personas in one place.

- **Load on use:** skills and agents enter the context only when needed.
- **Adapter:** MCP or JSON-RPC to servers, language servers and sandboxes.
- **Scale:** Hermes registers 69 tools but shows each platform a subset.

Takeaway: Registries keep the loop small while capabilities grow.

Reading anchor: Liu et al. (2025), Agent Design Pattern Catalogue, JSS 220, §4.16–4.17 · Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §8.

Next: Choosing patterns: the decision model — Tomás.

14. Choosing patterns: the decision model

Section: Topic 2 / Design patterns. **Presenter:** Tomás.

The catalogue's decision model asks one question per design problem.

Do: Read one row aloud as an example. Then hand over to Simon.

- **One question per row**, two answers.
- **Each answer has consequences** — benefits and costs.
- **A design** is a coherent set of answers.
- **Next:** how do we judge whether the choices are good?

Takeaway: A pattern is chosen by a question, not by fashion.

Reading anchor: Liu et al. (2025), Agent Design Pattern Catalogue, JSS 220, §5.1, Fig. 19.

Next: Why architecture evaluation must change — Simon.

15. Why architecture evaluation must change

Section: Topic 3 / Architecture evaluation. **Presenter:** Simon.

Classic scenario-based methods assume fixed requirements; agents decide much of their architecture at runtime.

Builds: press Right 4 times to complete the figure.

Do: Read each row left to right: classic assumption, then what an agent does instead.

- **Goals** are decomposed into sub-tasks while the agent runs.
- **The agent makes trade-offs** — inside guardrails we design.

- **Model, prompts and tools** change after evaluation.
- **New scenarios** are needed for memory, tools and guardrails.

Takeaway: Guardrails become the main architectural control point.

Reading anchor: Lu et al. (2026), AgentArcEval, JSS 232, §1.

Next: AgentArcEval: the process — Simon.

16. AgentArcEval: the process

Section: Topic 3 / Architecture evaluation. **Presenter:** Simon.

An ATAM-based method that evaluates agent architecture at design time and keeps re-evaluating at runtime.

Builds: press Right 3 times to complete the figure.

Do: Start at Understand; then the upper path, then the lower path and the loop back.

- **Understand first:** goals, governance, quality and guardrail requirements, the architecture.
- **Design time:** define and prioritise context scenarios.
- **Runtime:** monitor risks and reprioritise scenarios.
- **Output:** trade-offs, risks, improvements.

Takeaway: Agent architecture evaluation never really ends.

Reading anchor: Lu et al. (2026), AgentArcEval, JSS 232, §4, Fig. 2.

Next: A quality scenario in six parts — Simon.

17. A quality scenario in six parts

Section: Topic 3 / Architecture evaluation. **Presenter:** Simon.

Each scenario follows source → stimulus → environment → artefact → response → measure.

Builds: press Right 3 times to complete the figure.

Do: Walk the six parts left to right; stress that the measure can fail.

- **Source and stimulus:** who triggers what.
- **Artefact:** the part of the architecture under test.
- **Environment:** the conditions that make it hard.

- **Measure** makes the scenario checkable.

Takeaway: A scenario is only useful if its measure can fail.

Reading anchor: Lu et al. (2026), AgentArcEval, JSS 232, §5, §6.4.

Next: Eleven general scenarios and their measures — Simon.

18. Eleven general scenarios and their measures

Section: Topic 3 / Architecture evaluation. **Presenter:** Simon.

The catalogue gives one starter scenario per quality attribute; teams adapt and prioritise them.

Builds: press Right 2 times to complete the figure.

Do: Do not read all eleven; pick accuracy, safety and contestability, then reveal Luna's seven.

- **Eleven attributes**, from accuracy to contestability.
- **Each comes with a measure** a team can track.
- **Prioritising** is where the domain enters.
- **Luna tax copilot** chose 7 of the 11.

Takeaway: Prioritising scenarios is where the domain enters the evaluation.

Reading anchor: Lu et al. (2026), AgentArcEval, JSS 232, §5–6.

Next: Scenarios as a lens on real harnesses — Simon.

19. Scenarios as a lens on real harnesses

Section: Topic 3 / Architecture evaluation. **Presenter:** Simon.

The same scenario catalogue exposes where real harnesses put their architectural effort.

Do: Read the Safety row across all three columns, then one more row. Hand over to Luca.

- **Rows** are AgentArcEval attributes; cells are architectural mechanisms.
- **Pi** optimises for a person at a terminal.
- **Pi Durable** optimises for availability.
- **Hermes** invests in safety policy and self-improvement.

Takeaway: A scenario catalogue makes the trade-offs between harnesses visible.

Reading anchor: Lu et al. (2026), AgentArcEval, JSS 232 · our mapping from Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, Earendil (2026), Pi Durable, earendil.com/posts/pi-durable.

Next: Context is a budget — Luca.

20. Context is a budget

Section: Topic 4 / Context engineering. **Presenter:** Luca.

Context engineering decides what task-relevant information the model has — and what it does not.

Builds: press Right 2 times to complete the figure.

Do: Point at the two threshold lines, then the three ways to free space.

- **Prompt engineering:** how a task is described.
- **Context engineering:** what information the model can access.
- **The window fills** with every tool output.
- **Compaction** frees space — at different thresholds and in different ways.

Takeaway: Every token in the window competes for the model's attention.

Reading anchor: Mohsenimofidi et al. (2026), Context Engineering for AI Agents in OSS, MSR '26, §1 · Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §7.

Next: Who actually writes context files? — Luca.

21. Who actually writes context files?

Section: Topic 4 / Context engineering. **Presenter:** Luca.

An empirical study scanned 10,000 popular GitHub projects for AI context files.

Builds: press Right 3 times to complete the figure.

Do: Reveal the circles from largest to smallest: 10,000, then 466, then 155.

- **Formats:** Copilot instructions, CLAUDE.md, AGENTS.md, GEMINI.md.
- **Adoption is early:** only 5% of mature projects.
- **AGENTS.md** is the tool-agnostic convention, so it is studied in depth.
- **Most common pair:** AGENTS.md with CLAUDE.md.

Takeaway: Context engineering is real practice, but still early.

Reading anchor: Mohsenimofidi et al. (2026), Context Engineering for AI Agents in OSS, MSR '26, §3–4.1.

Next: What goes into AGENTS.md — Luca.

22. What goes into AGENTS.md

Section: Topic 4 / Context engineering. **Presenter:** Luca.

Projects describe conventions, contribution rules and architecture — rarely security.

Builds: press Right 2 times to complete the figure.

Do: Point at the top bar, then at the security bar at the bottom.

- **Top three:** conventions, contribution guidelines, architecture.
- **Then:** build and test commands — what an agent must run.
- **Rare:** troubleshooting and security.
- **No shared structure** yet across projects.

Takeaway: Teams document how to work here, not how to stay safe.

Reading anchor: Mohsenimofidi et al. (2026), Context Engineering for AI Agents in OSS, MSR '26, §4.2, Table 1.

Next: Five ways to phrase an instruction — Luca.

23. Five ways to phrase an instruction

Section: Topic 4 / Context engineering. **Presenter:** Luca.

The same convention can describe, order, forbid, explain or set a condition.

Builds: press Right 2 times to complete the figure.

Do: Read two quotes aloud: one prescriptive, one prohibitive.

- **Descriptive** states facts; **prescriptive** gives orders.
- **Prohibitive** sets boundaries — "never …".
- **Explanatory** adds the reason; **conditional** a situation.
- **Open question:** which style changes agent behaviour most?

Takeaway: How you phrase context is a design choice, not a detail.

Reading anchor: Mohsenimofidi et al. (2026), Context Engineering for AI Agents in OSS, MSR '26, §4.2.

Next: Context files are maintained artifacts — Luca.

24. Context files are maintained artifacts

Section: Topic 4 / Context engineering. **Presenter:** Luca.

Half the files never change; active ones are tuned instruction by instruction.

Builds: press Right 2 times to complete the figure.

Do: Point at the crimson half of the squares first, then the right chart.

- **50% unchanged** since they were added.
- **Active files:** mostly add or reword instructions.
- **Few removals:** files tend to grow.
- **Treat them like code:** versioned, reviewed, tested.

Takeaway: Context files co-evolve with the code they describe.

Reading anchor: Mohsenimofidi et al. (2026), Context Engineering for AI Agents in OSS, MSR '26, §4.3, Table 2.

Next: What stays in context, what waits on disk — Luca.

25. What stays in context, what waits on disk

Section: Topic 4 / Context engineering. **Presenter:** Luca.

Harnesses keep the always-loaded context small and move the rest behind a tool call.

Builds: press Right 3 times to complete the figure.

Do: Start above the waterline, then go deeper: on demand, then on disk.

- **Always loaded:** instructions, schemas, a skill catalog, small memory files.
- **On demand:** a skill's full text only when it is used.
- **On disk:** full history, searchable but out of the window.
- **Hermes caps memory** at a few thousand characters on purpose.

Takeaway: Load context just in time, not just in case.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §7, §15 · Nous Research, Hermes Agent docs; Alejandro AO (2026), Hermes architecture.

Next: Subagents are context engineering — Luca.

26. Subagents are context engineering

Section: Topic 4 / Context engineering. **Presenter:** Luca.

A subagent spends tokens exploring in its own context and returns one short report.

Builds: press Right 3 times to complete the figure.

Do: Follow the fan-out to the workers and back to the single tool result. Then hand over to Matheus.

- **The orchestrator** delegates several prompts at once.
- **Workers** start with a fresh context and keep their search noise.
- **The parent keeps** only the call and the reports.
- **Hermes** runs one child by default; in Pi Durable a subagent is a conversation owned by a tool call.

Takeaway: The harness decides what the orchestrator never sees.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §11, Rec. 12 · Earendil (2026), Pi Durable, earendil.com/posts/pi-durable.

Next: From session agent to always-on agent — Matheus.

27. From session agent to always-on agent

Section: Topic 5 / From sessions to always-on agents. **Presenter:** Matheus.

The same loop can run per session, survive crashes, or run all the time — each step adds state and responsibility.

Builds: press Right 3 times to complete the figure.

Do: Climb the stairs one step at a time; stress that this spectrum is our own synthesis.

- **Pi:** a person at a terminal starts and supervises each run.
- **Pi Durable:** a library for apps whose agents must survive restarts.
- **Hermes:** started by chats and schedules, not only by people.
- **What grows:** state, triggers and the surfaces that reach the agent.

Takeaway: Always-on is an architectural decision, not a feature flag.

Reading anchor: Zechner (2025); Pi docs, github.com/earendil-works/pi · Earendil (2026), Pi Durable, earendil.com/posts/pi-durable · Nous Research, Hermes Agent docs; Alejandro AO (2026), Hermes architecture.

Next: Pi Durable: surviving a crash — Matheus.

28. Pi Durable: surviving a crash

Section: Topic 5 / From sessions to always-on agents. **Presenter:** Matheus.

Every step is a task that stores a checkpoint before it moves on.

Builds: press Right 3 times to complete the figure.

Do: Upper lane, then the storage bar, then the lower lane after the restart.

- **Checkpoint first:** a tool call's intent is stored before it runs.
- **After a restart**, a new process resumes open tasks.
- **Replay rule:** only tools marked safe rerun; otherwise the model is told.
- **requestId** makes a submission exactly-once.

Takeaway: Durability means deciding, per tool, what may happen twice.

Reading anchor: Earendil (2026), Pi Durable, earendil.com/posts/pi-durable.

Next: Hermes: an always-on agent — Matheus.

29. Hermes: an always-on agent

Section: Topic 5 / From sessions to always-on agents. **Presenter:** Matheus.

One agent core, reached through a CLI, chat gateways and editors, and started by a scheduler.

Builds: press Right 3 times to complete the figure.

Do: Entry points first, then cron, then the four boxes on the right.

- **Entry points:** CLI, a long-running gateway, ACP clients.
- **Cron** starts work without a person in the loop.
- **Memory files** are always in context; sessions live in SQLite.
- **Self-improving:** the agent writes and patches its own skills.

Takeaway: An always-on agent is a service, with a service's responsibilities.

Reading anchor: Nous Research, Hermes Agent docs; Alejandro AO (2026), Hermes architecture · Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §7–10.

Next: What always-on costs — Matheus.

30. What always-on costs

Section: Topic 5 / From sessions to always-on agents. **Presenter:** Matheus.

More autonomy moves risk from the person at the terminal into the architecture.

Do: Read the last row across all three columns.

- **Each step right** adds state that must stay consistent.
- **Unattended runs** need durable checkpoints and replay rules.
- **Long-running processes** fail in new ways: leaks, replays, fragmentation.
- **Field reports** are evidence too: read the issue tracker.

Takeaway: Every hour an agent runs unattended is an hour of architecture to own.

Reading anchor: Zechner (2025); Pi docs, github.com/earendil-works/pi · Earendil (2026), Pi Durable, earendil.com/posts/pi-durable · Hermes issue #5563.

Next: The platform turn — Matheus.

31. The platform turn

Section: Topic 5 / From sessions to always-on agents. **Presenter:** Matheus.

Harnesses are becoming platforms, and frameworks are becoming harnesses.

Builds: press Right 2 times to complete the figure.

Do: Read the plates bottom up; name one logo per layer.

- **Bottom up:** model, harness, platform, meta-harness.
- **Platform:** SDKs, durable libraries, ACP servers, skill marketplaces.
- **Meta-harness:** Omnigent ships adapters for 23 harnesses.
- **Implication:** extensions are a supply chain to secure.

Takeaway: The developer increasingly programs inside the harness.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §14 · Earendil (2026), Pi Durable, earendil.com/posts/pi-durable.

Next: Recommendations from eleven harnesses — Matheus.

32. Recommendations from eleven harnesses

Section: Topic 5 / From sessions to always-on agents. **Presenter:** Matheus.

The study ends with 18 recommendations; seven connect directly to today's topics.

Do: Pick R1, R8 and R18; ask the room which one they would break.

- **Recommendations** come from what eleven systems converged on.
- **Evidence column:** where today's systems follow or diverge.
- **R18 splits the field:** budgets versus trusting the model to stop.
- **Use them as defaults**, and know when you diverge.

Takeaway: Recommendations are evidence, not rules — know when you diverge.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §16.

Next: A minimum viable harness in ~90 lines — Matheus.

33. A minimum viable harness in ~90 lines

Section: Topic 5 / From sessions to always-on agents. **Presenter:** Matheus.

The study closes with a scaffold that implements ten of its recommendations.

Do: Point at the while loop, then at the return line.

- **The while loop** is the whole agent.
- **Limits and compaction** run before every model call.
- **Return when** the model stops calling tools.
- **Left out on purpose:** sandbox, multi-agent, MCP, skills.

Takeaway: Start minimal; add what your observed failures demand.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §16.10, Listing 3.

Next: Discussion — All presenters.

34. Discussion

Section: Discussion. **Presenter:** All presenters.

Which of today's five questions can our own project already answer?

Sources

- Barbaste, Darrigol, Vu & Wiltberger (2026). *Harness Engineering: Anatomy, Architecture, and Evolution of Coding Agents.* arXiv:2609.00006.
- Liu, Lo, Lu, Zhu et al. (2025). *Agent Design Pattern Catalogue.* Journal of Systems and Software 220. arXiv:2405.10467.
- Lu, Zhao, Liu, Zhang et al. (2026). *AgentArcEval: An Architecture Evaluation Method for Foundation Model based Agents.* Journal of Systems and Software 232. arXiv:2510.21031.
- Mohsenimofidi, Galster, Treude & Baltes (2026). *Context Engineering for AI Agents in Open-Source Software.* MSR '26. arXiv:2510.21413 (CC BY 4.0).
- Earendil (2026). *Pi Durable.* earendil.com/posts/pi-durable (1 October 2026); Pi source: github.com/earendil-works/pi.
- Nous Research. *Hermes Agent* documentation, hermes-agent.nousresearch.com; Alejandro AO (2026), *Hermes Agent Architecture Explained*, alejandro-ao.com.
- Further reading: Koenigstein (2026), *Harness Engineering for AI Agents*; Grootendorst & Alammar, *An Illustrated Guide to AI Agents*; Zechner (2025), "What I learned building an opinionated and minimal coding agent"; Ronacher (2026), "Pi: The Minimal Agent Within OpenClaw".

All diagrams are original drawings in the lecture's design system; where a diagram simplifies a paper figure, the slide says so.